多奇·教育訓練

2026-05-11 | 永豐銀行

規格驅動開發實戰

AI 時代的軟體開發新典範 — 以 GitHub Spec Kit 為例

多奇數位創意有限公司

技術總監 黃保翕（Will 保哥）

https://blog.miniasp.com







SDD and Spec Kit

簡介 SDD 與 Spec Kit

簡介 Spec-Driven Development (SDD)

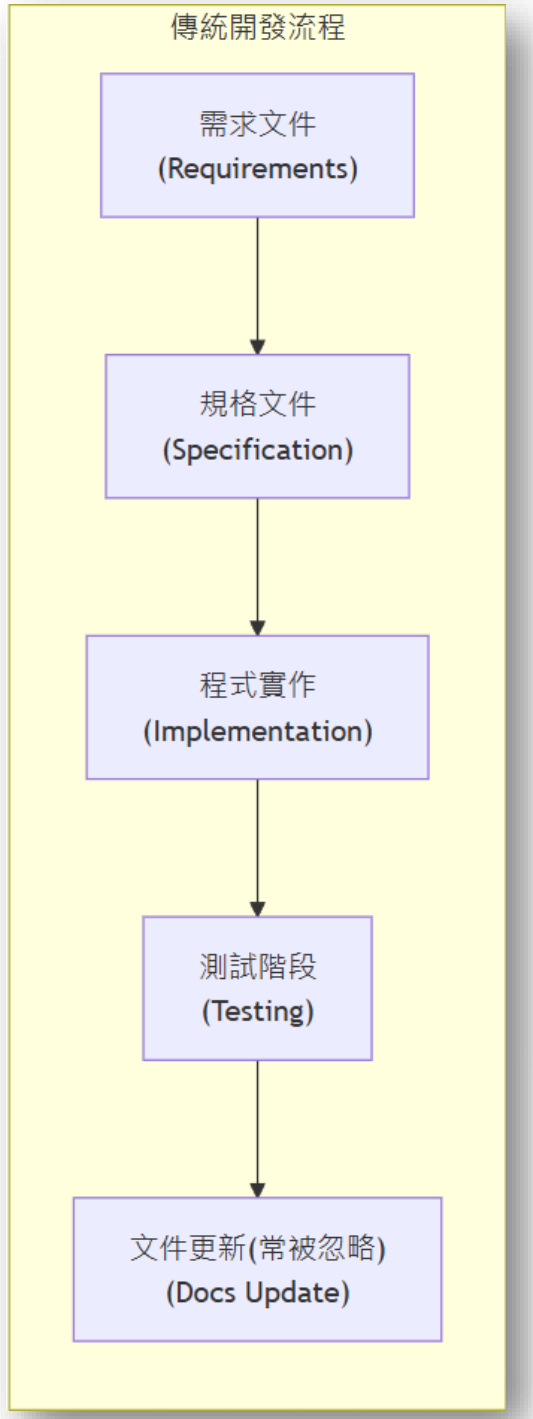
- **Spec-Driven Development** (**SDD**) 是一套**方法論** (Methodology)
  - 一種把「**規格**」變成「**開發核心產物**」的**軟體開發方法**
  - 它反轉了傳統流程裡「先寫程式、文件只是附屬說明」的慣例
  - 在 SDD 裡，**規格是主角**，**程式只是它的投影**！
- **傳統開發是**
  - **需求** → 設計 → 文件 → 程式碼
  - 實際維護時，**程式碼才是唯一真相**（人會跑、文件沒人更新、註解工程師懶得改）
- **SDD 的想法是**
  - **需求** → **規格** → 實作計畫 → 任務 → 程式碼
  - 程式碼由規格生成，更新規格即可再生實作
  - **規格是可執行的文件**， AI 會讀懂它、規劃架構、拆解任務、產出程式。

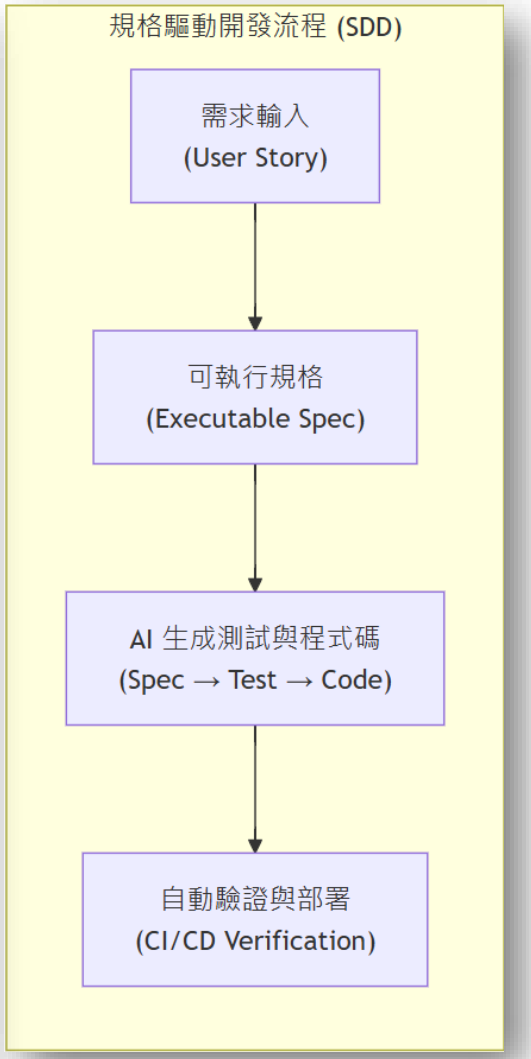
多奇·教育訓練

傳統開發 vs. SDD

- **傳統開發流程**
  - 需求文件 → 規格文件 → 程式實作 → 測試階段 → 文件更新（被遺忘）
- **SDD 開發流程**
  - 需求輸入 → 可執行規格 → AI 生成測試與程式碼 → 自動驗證與部署

> 傳統開發流程中，規格只是過場文件。
>
> 在 SDD 流程中，規格是整個開發的中樞神經。





多奇·教育訓練

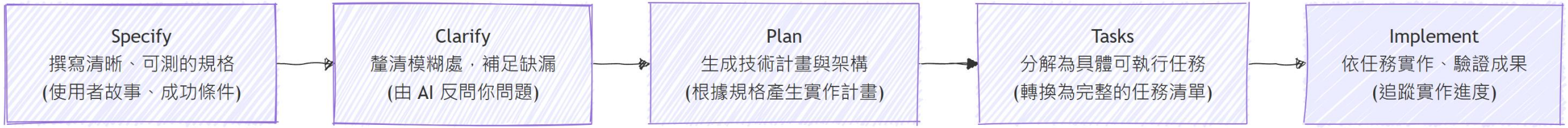
簡介 GitHub Spec Kit

- **Spec Kit** 則是一套協助你**實現 SDD 方法論**的**完整工具組**
  - 由 **GitHub** 主導發展，支援市面上所有主流 AI Coding 工具
  - 它把開發流程的主角從「程式碼」翻轉成「規格」
  - 它讓規格文件成為可執行、可生成實作的核心產物

- 🌱 核心理念
  - **意圖導向開發**：先定義「**做什麼**」（what），再決定「**怎麼做**」（how）
  - **豐富規格撰寫**：結合防呆機制與組織原則
  - **多步驟的精煉**：避免單次提示直接產生程式碼
  - **高度依賴先進 AI 模型**對**規格**的**解讀能力**

多奇·教育訓練

GitHub Spec Kit 的基本流程



- **Specify**
  - 撰寫清晰、可測的規格，定義使用者故事，定義成功條件，給予明確的目標。將需求拆成結構化 Prompt！
- **Clarify**
  - 釐清模糊處，補足缺漏，透過 AI 對你靈魂烤問，幫助你思考需要釐清的地方
  - 讓 AI 幫你審查規格而非直接寫程式
- **Plan**
  - 生成技術計畫與架構，根據規格產生實作計畫
- **Tasks**
  - 分解為具體可執行的任務，將規格轉換為完整的任務清單
- **Implement**
  - 依任務實作、驗證成果，透過一系列檢核清單 (Checklist) 追蹤實作進度

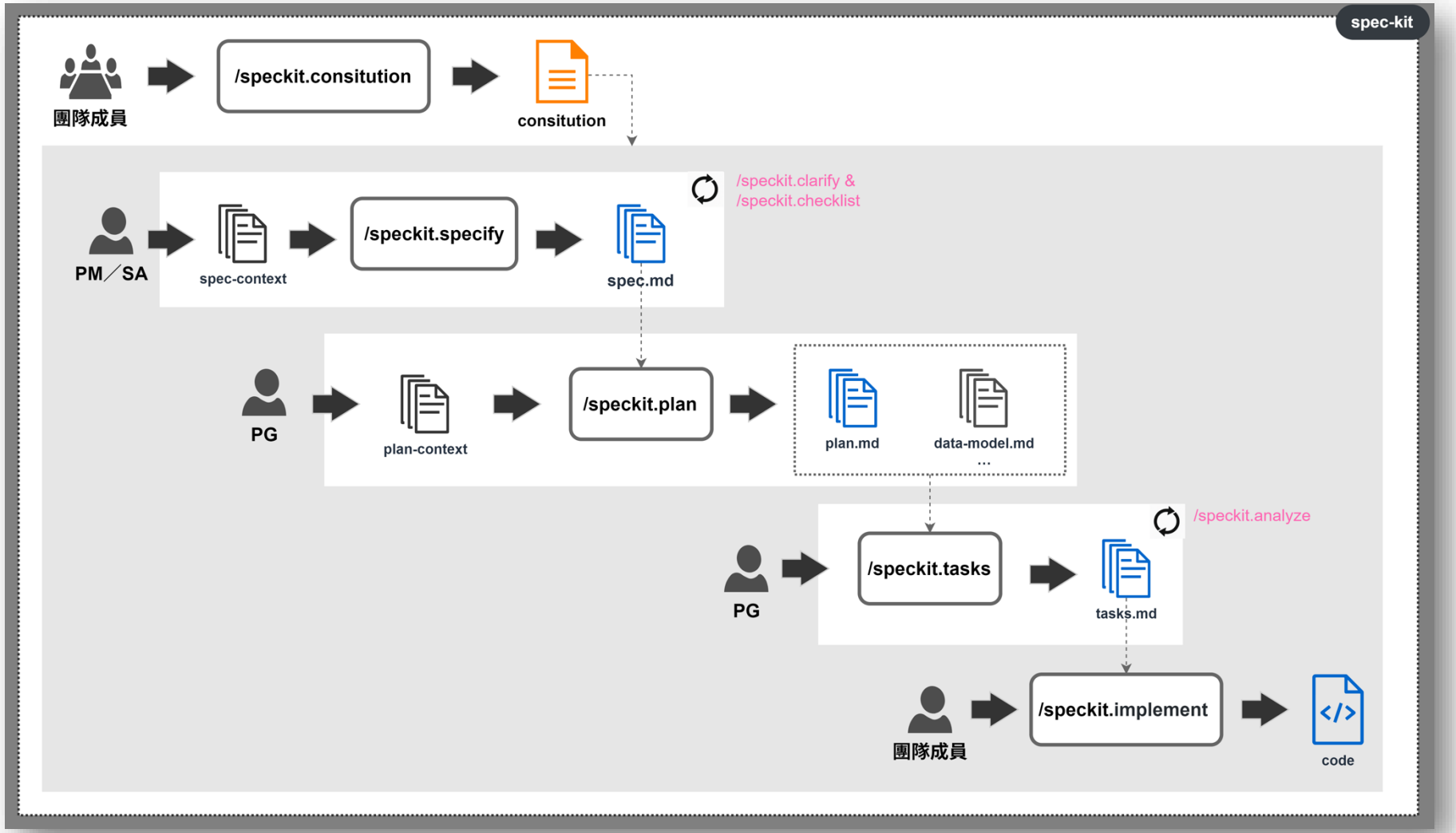
多奇·教育訓練

適用於 Spec Kit 的開發階段

階段	重點	主要活動
全新專案 (Greenfield)	從零開始產生	- 從高階需求開始 - 產生規格文件 - 規劃實作步驟 - 建構可上線的應用程式
現有專案 (Brownfield)	舊系統現代化	- 逐步新增功能 - 現代化既有系統 - 調整流程
創意探索	平行實作	- 探索多元解法 - 支援多種技術堆疊與架構 - 實驗不同 UX 模式

多奇·教育訓練

從規格到程式碼的大致流程（SDD + Spec Kit）



多奇·教育訓練

GitHub Spec Kit 與 AI Coding 工具的應用實例

- **原始規格**

> # 會員註冊流程
>
> 我們的核心目標是讓新使用者能透過一個簡單、安全的流程，順利註冊帳號並完成 E-Mail 驗證，以便使用平台功能。
>
> 整個註冊是一個單向流程，無法返回上一步。使用者首先需要填寫身分證字號、姓名等資料，並設定一組 8 到 20 碼、包含英文大小寫與數字的密碼。系統會檢查身分證字號，防止重複註冊。
>
> 送出資料後，系統會寄出一封內含 6 位數驗證碼的 E-Mail，有效時間為 5 分鐘。使用者必須手動輸入驗證碼來完成驗證。
>
> 如果使用者未完成 E-Mail 驗證，依然可以用剛設定的帳號密碼登入，但部分功能將受限。直到輸入正確的驗證碼後，帳號所有功能才會被啟用。註冊完成後，系統不會自動登入。
>
> 為了讓流程單純，我們不做社群帳號註冊、不提供密碼強度提示，也不支援點擊 E-Mail 連結直接驗證。

- **完成作品 (相同的規格，不同的實作，沒寫一行 Code 完工)**
  - https://github.com/doggy8088/duotify-membership-v1 (.NET 8)
  - https://github.com/doggy8088/duotify-membership-v2 (Node.js)
  - https://github.com/doggy8088/duotify-membership-v3 (Go)
  - https://github.com/doggy8088/duotify-membership-v4 (Python)



The evolution of development processes

從傳統到規格驅動：開發流程的演進

為什麼傳統文件與程式碼總是脫節

- **主要觀點**
  - **文件過時**，**程式失真**。我們一直在**追趕需求的影子**。
- **重點觀念**
  - 為什麼文件總是落後？因為「寫程式」比「寫文件」爽快！
    - (寫程式可以立刻看結果，寫規格不行)
  - DevOps 解決部署痛點，卻沒解決「溝通斷層」
    - (我們只是把速度加快了，沒有改善根本問題) (因為大部分人覺得無解)
  - 敏捷讓交付更快，但沒有讓「理解」更準
    - (敏捷加快了「做」的速度，卻常忽略「為什麼這麼做」的理解)
  - **SDD** 的出現，是把「規格」從文件變成引擎。
    - (程式碼只是規格的最後一哩路！)
- **互動問題**
  - 你上次打開規格文件是什麼時候？它的內容還對得上實際產品嗎？

多奇·教育訓練

敏捷與 DevOps 解了什麼、沒解什麼？

面向	敏捷/DevOps 成果	尚未解決
交付速度	✅ 快速迭代、持續整合	❌ 理解偏差仍高
品質控制	✅ 自動測試、監控	❌ 規格脫節
團隊文化	✅ 擁抱變化	❌ 仍靠口頭共識

> 我們變得更快，但並沒有更準。

多奇·教育訓練

為什麼我們總是在「重複同樣的錯」？

- 文件寫完沒人看
- 程式寫完規格就過期
- 專案越大，落差越大
- 敏捷讓交付更快，卻沒讓理解更清楚

> **我們其實一直在修理「溝通」這台壞掉的機器。每次加一個流程、換一個工具，但核心問題——規格與程式的距離——始終沒變。**

多奇·教育訓練

什麼是 Spec-Driven Development？

> Spec-Driven Development (SDD) 方法論顛覆了傳統軟體開發的流程。
>
> 數十年來，程式碼一直是主角 —— 規格僅僅是我們搭建並在「真正開始寫程式」後就丟棄的鷹架。
>
> Spec-Driven Development 則不同：**規格本身可以執行**，不只是作為參考指引，而是能直接產生可運作的實作。
>
> — GitHub Spec Kit

多奇·教育訓練

SDD (規格驅動開發) 的精神轉向

- 從「程式碼為中心」到「規格為真相」
  - 程式碼只是結果，不是意圖
  - 規格是唯一可信的事實來源
  - AI 讓規格成為「可執行」的橋樑

> **開發的真相不再藏在程式裡，而是寫在規格裡。**

多奇·教育訓練

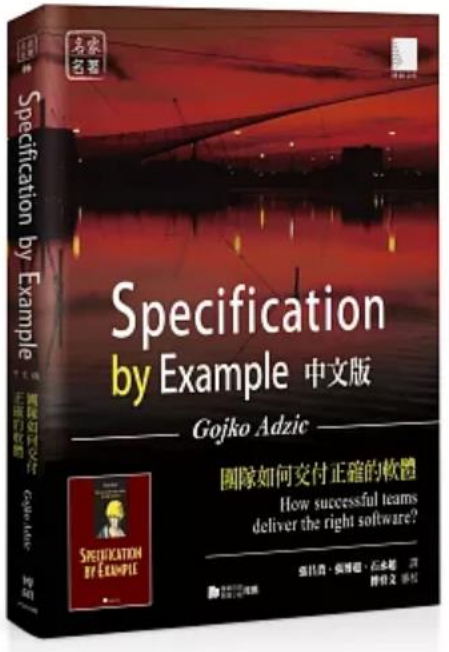


Creating executable specification documents

規格即真相：建立可執行的規格文件

規格不是文件，是指令

- 一份好的規格**能被測試**、**能被執行**、**能驅動程式碼**
- **每一條規格**都該有**驗收條件** (Acceptance Criteria)
  - 要學習如何撰寫 User Story (使用者故事)
- **「範例優於描述」——不要形容，要舉例！**
  - Specification by Example (SBE)
  - 行為驅動開發 (Behavior-Driven Development) (BDD)



此書已絕版

> **「規格」不是**說明書，**而是行為合約。**

多奇·教育訓練

模糊與可執行規格的對比

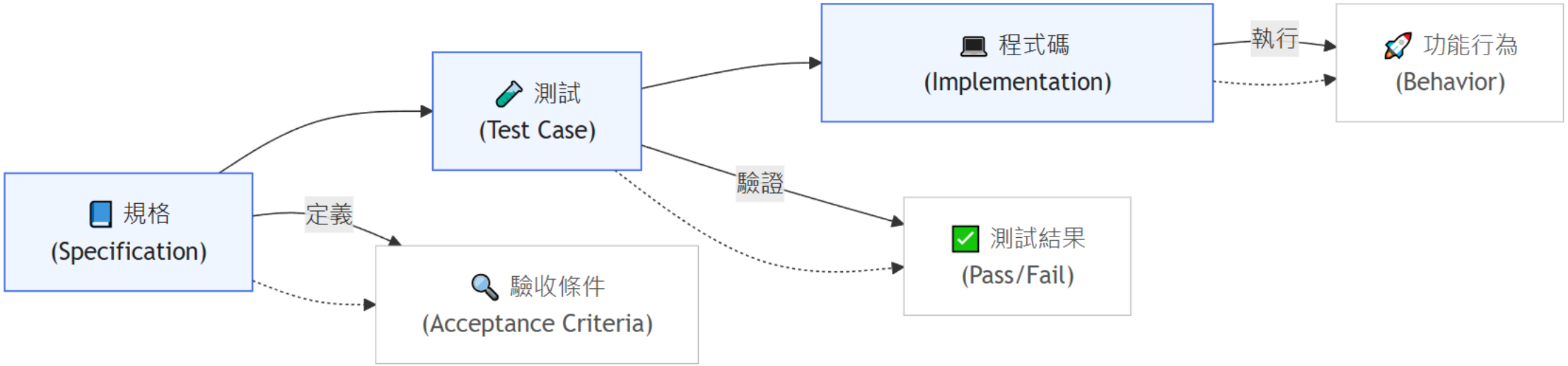
模糊描述	可執行規格
系統應提供安全的登入機制	當使用者輸入錯誤密碼三次時，鎖定帳號 5 分鐘
頁面要美觀易用	按鈕距離表單 24px，字體使用 16px Sans

> 模糊規格讓人爭論，可執行規格讓 AI 行動。

多奇·教育訓練

規格與測試的一致性

- 每條規格都能自動生成測試案例
- 驗收條件可直接對應測試名稱
- 測試通過 = 規格被滿足



多奇·教育訓練



From specification to implementation

AI 與開發的結合：從規格到程式碼

AI 在規格展開、技術研究與設計決策中的角色

面向 Orient	規格展開 Specification Expansion	技術研究 Technical Research	設計決策 Design Decision
核心任務	將人類意圖展開成具體、可驗證的需求	探索未知技術與可行方案	評估方案、模擬後果並輔助取捨
主要輸出	**spec.md** 使用者故事、功能需求、成功準則	**research.md** 技術比較、替代方案、依據	**plan.md** 技術選擇、架構、約束條件
人類的責任	定義願景與價值、確認邊界	驗證研究方向與可信來源	做出最終取捨與背書理由
AI 的角色定位	**將自然語言轉成結構化規格** 結構化編輯者/語意編譯器/生成範例	**協助做技術研究與模組規劃** 知識探索代理 (research agent)	**協助模擬設計方案** 策略顧問與衝突檢查者/補完邏輯
AI 的能力焦點	語意解析/行為拆解/可測試性檢查	資訊蒐集/模式比對/風險分析	模擬架構選項/偵測衝突/生成平行方案
常見互動型態	「這個需求夠具體嗎？」→ 提出澄清問題	比較 A 與 B 在延展性與維護性上的差異	如果選 B，性能與複雜度的代價是？
品質驗證機制	檢查模糊詞、需求覆蓋率、可量化指標	確保研究來源與假設透明	檢查是否違反憲章原則 （如簡潔、可測試）
典型錯誤風險	過度自動化 → 缺乏人類意圖校正	混合主觀推測與事實	AI 模擬結果被誤認為真實決策
價值貢獻	提升需求明確度與一致性	降低技術探索成本、建立知識脈絡	促進跨角色對話與設計理性化
關鍵問題	「我們到底想達成什麼？」	「這樣做可行嗎？」	「哪個選項最符合原則與現實？」

AI 生成不是魔術：如何避免「幻覺」與不精準

- **AI 不會「知道」，它只會「預測」**
  - 大多 LLM 並非知識庫，而是語言預測引擎（**但可以藉由搜尋引擎補充缺少的知識**）
  - 它不是查資料，而是根據「過去見過的語言模式」來生成可能的句子（**很會寫程式無庸置疑**）
  - 所以它說話像懂很多，但事實上只是「很會接話」（**很容易過度設計與補充不存在的資訊**）
- **什麼是「幻覺」？**
  - AI 生成出「看似合理但錯誤」的內容（**因為 AI 的邏輯很好，常會腦補「合理」的東西進來**）
  - 常見類型
    - **捏造資料**：虛構人名、事件、文件
    - **錯誤歸因**：把別人的理論說成某位學者提出
    - **邏輯跳躍**：推論過程缺步、結論自說自話
    - **語氣誤導**：以肯定語氣表達不確定內容

多奇·教育訓練

如何降低幻覺風險

- **基本策略**
  - **給明確上下文**：指明資料、來源、格式、方案、套件、程式語言、框架、...
  - **要求引用依據**：例如「附上出處或段落」或「僅使用我提供的文件」
  - **設立檢查關卡**：用第二個模型交叉驗證、TDD、BDD、SBE
  - **避免過度壓縮**：摘要過度容易遺失關鍵事實、AI 自動壓縮功能盡量不要開
  - **定義容錯邊界**：哪些錯誤可以接受、哪些不行、提供邊緣案例
- **補充說明**
  - 幻覺可能不會消失，但可以被「限制在安全範圍內」
  - 再聰明的大語言模型，也會有寫錯的機率
  - 不要抱怨！再試一次就好！🤘

多奇·教育訓練

“

AI 是副駕，不是法官

讓它輔助你判斷，而不是替你思考

”



Constitutional Principles

憲法式原則

憲章基礎：強化架構紀律

SDD 方法論

- **SDD 的核心是一份憲章 (Constitution)**
  - 定義團隊的**開發信念**與**不可退讓**的原則
  - 定義一套**絕對不可變更**的**原則** (有這種東西嗎?)，規範規格如何轉化為程式碼
  - 這份憲章就像**系統的架構 DNA**，確保每一次產生的實作都能維持**一致性**、**簡潔性**與**品質**
- **開發的九大條款 !?**
  - 條款一：以函式庫為先 (Library-First) 原則
  - 條款二：命令列介面 (CLI Interface) 強制規定
  - 條款三：測試優先原則 (Test-First Imperative)

    Articles IV, V and VI are missing #888
  - 條款七與條款八：簡單性與反抽象
  - 條款九：以整合為優先的測試 (Integration-First Testing)

多奇·教育訓練

模組化、簡單化、可測試的核心規範

- **官方的範例**

  `/speckit.constitution` Create principles focused on **code quality**, **testing standards**, **user experience consistency**, and **performance requirements**

- **還可以加什麼？**
  - Create principles focused on **clarity over cleverness**, **readability over optimization**.
  - Mandate **Test-Driven Development** (**TDD**) for **all new features**.
  - Require **structured logging** and **traceable metrics** in every service.
  - Require **minimum viable complexity** — if a **simpler approach** works, adopt it first.
  - Enforce **library-first** architecture — each feature must start as a **self-contained module or library**.
  - Prioritize **intent-driven development** — every line of code should **trace back** to a **user or business intent**.

- **簡單公式**
  - **原則類型**（技術 / 文化）＋ **關鍵價值**（模組化、可測試、可維護）＋ **不可違背的規範**（MUST/SHOULD）

多奇·教育訓練

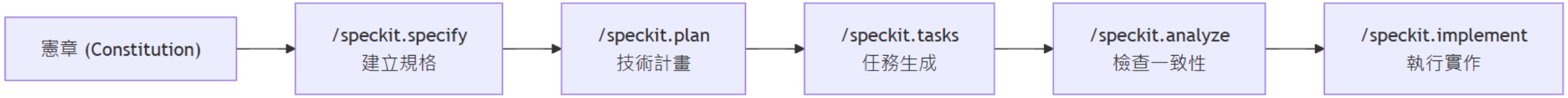
如何讓 AI 生成代碼符合架構守則

1 先給「憲章」，再給「任務」

- 提供**上下文連貫**且**符合邏輯**的憲章（所以團隊中有誰能寫出符合邏輯的憲章？）

2 將原則寫成「可驗證規範」

- **模糊的價值觀**會**被忽略**，**可執行的規範**才能**被遵守**。
- **模組化**：「每個功能需為獨立模組」
- **測試性**：「所有功能以 TDD 方式建立，測試先行」
- **簡單化**：「禁止建立超過三層封裝的抽象」
- **可觀察性**：「每個模組必須輸出結構化 log」



多奇·教育訓練

憲章會在每個階段加入自動稽核

階段	AI 任務	驗證目標
/speckit.specify	審查需求是否違反憲章	規格清晰且可測試
/speckit.plan	核對架構邏輯與原則一致	不違反模組化、簡單化
/speckit.tasks	任務分解符合測試流程	有明確輸入與輸出
/speckit.analyze	跨文件一致性分析	規格、計畫、任務三者對齊

多奇·教育訓練

讓憲章具「法律效力」

- 在憲章內明定
  - **MUST**：必須遵守（違反即 Critical Error）
  - **SHOULD**：建議遵守（允許有理由的例外）

- 每次透過 **`/speckit.analyze`** 分析會自動產出多份精簡分析報告
  - 規格分析報告 (Specification Analysis Report)
  - 覆蓋率摘要表 (Coverage Summary Table)
  - 憲章對齊問題 (Constitution Alignment Issues)
  - 未對應任務 (Unmapped Tasks)
  - 各類指標 (Metrics)

多奇·教育訓練

人類角色的轉變

角色	職責
架構師	制定憲章與原則
所有人	撰寫規格、解釋例外、腦力激盪、通靈思考
工程師	制定技術規格、維護架構、問題分析、Code Review、Debug、... (Coding?)
AI Agent	自動生成程式碼、執行守則、回報違規

> AI 代碼不是自動生成的魔術，
> 而是一場「由原則驅動」的自動化。

多奇·教育訓練

防止「過度設計」與「技術債」

用規範

多讀書

先交付

慎擴充

慎選框架

Code Review

常重構

少預想

拆小步

勇刪程式

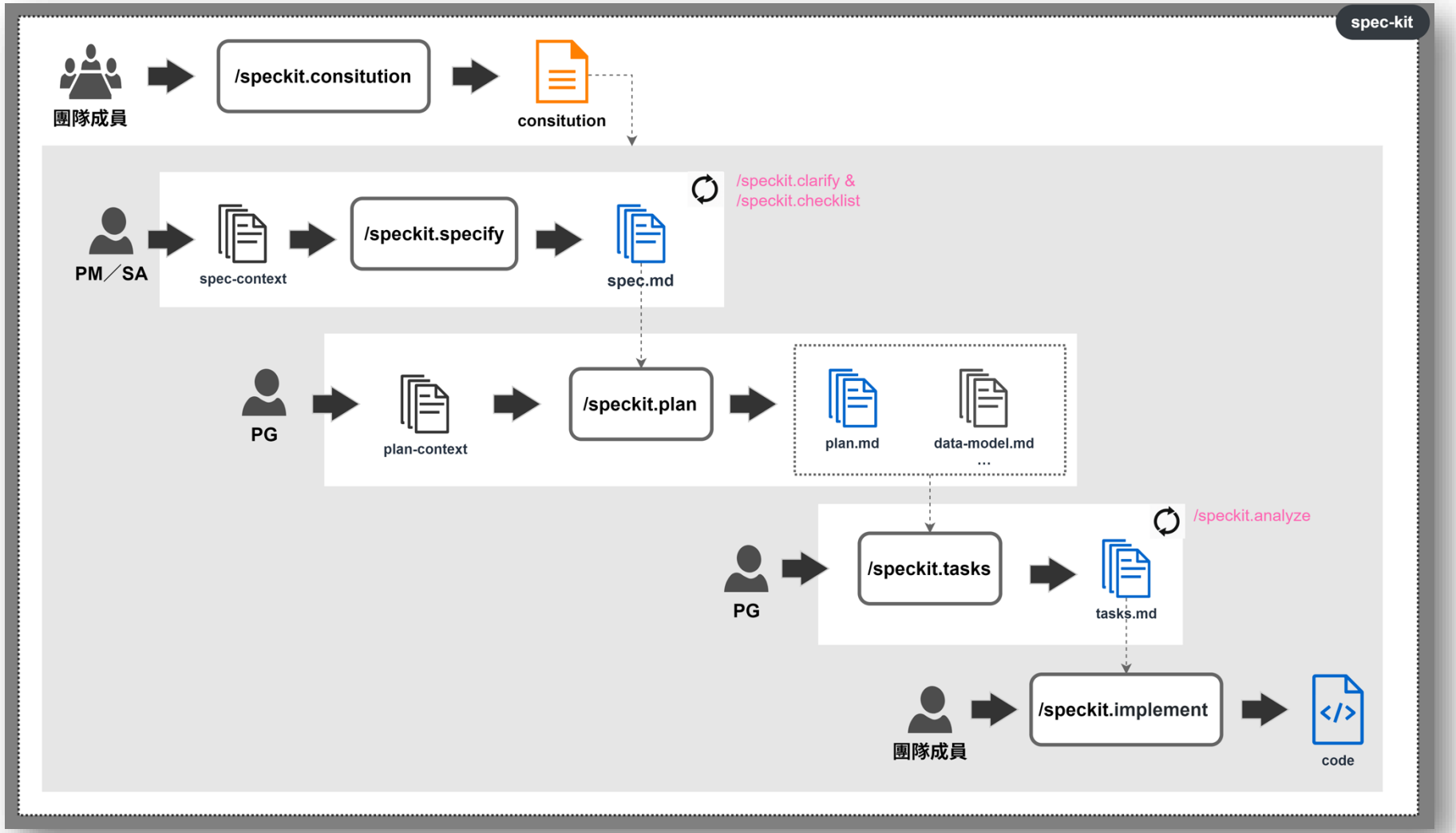
多奇·教育訓練



規格生成 → 實作規劃 → 任務拆解

工作流程全覽

從規格到程式碼的大致流程



多奇·教育訓練

實際操作演練

- 從一個需求到完整的專案實作
- 示範如何從規格直接生成測試案例
- 示範如何透過 TDD 驅動程式碼驗證
- 📚 **學員實作手冊**
  - https://sdd.gh.miniasp.com/labs/
- 🤖 **透過 AI Agent 自動進行整個 SDD 開發流程**
  - https://sdd.gh.miniasp.com/labs/ai-agent
- 📄 **Spec Kit Constitution Generator (Claude Projects)**
  - https://sdd.gh.miniasp.com/labs/claude-projects

多奇·教育訓練

認識 Spec Kit 與 Specify CLI

面向	Spec Kit	Specify CLI
用途	「理念與模板」— 定義規格驅動開發的標準	「執行引擎」— 讓你在終端或 AI 代理中跑起 Spec Kit 流程
定位	一整套 規格驅動開發 (SDD) 的框架與範本	Spec Kit 的指令列介面，用來啟動與操作整個流程
主要功能	提供模板、流程與方法論：包含 spec、plan、tasks、constitution 等完整架構	初始化專案、產生模板、呼叫 AI 代理人 執行 **/speckit.*** 命令
核心組件	1. /templates/ 模板庫 2. /memory/constitution.md 專案憲章 3. SDD 方法論文件與教學	1. specify 命令（Python CLI） 2. 子命令：init、check、plan 等 3. 透過腳本 (.sh 與 .ps1) 自動生成各階段文件
工作範疇	理論與架構層： 定義 SDD 流程、原則、模板結構與憲章制度	實作層： 讓使用者在終端機或 AI 開發環境中執行這些流程
輸出成果	一組標準化檔案與流程規範 例如：docs、templates、memory 資料夾	在本地建立完整 Spec Kit 專案結構， 並連接 AI 代理進行互動式生成
與 AI 的關係	提供給 AI 代理使用的框架與指令集	呼叫 AI 代理並注入 Spec Kit 命令與模板
使用場景	組織導入 SDD 時的核心標準與教學資源	實際在本機或雲端環境建立與執行 Spec Kit 專案

多奇·教育訓練

Spec Kit 支援的 AI agents

Agent	支援狀態	備註
GitHub Copilot	✅	
Claude Code	✅	
Gemini CLI	✅	
Cursor	✅	
Qwen Code	✅	
opencode	✅	
Windsurf	✅	
Kilo Code	✅	
Auggie CLI	✅	
CodeBuddy	✅	
Roo Code	✅	
Codex CLI	✅	
Amazon Q Developer CLI	⚠️	Amazon Q Developer CLI does not support slash 指令的自訂參數。
Amp	✅	
SHAI (OVHcloud)	✅	

https://github.com/github/spec-kit/

多奇·教育訓練

⚡ 快速開始｜安裝 Specify CLI

- 持久安裝

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
specify init <PROJECT_NAME>
specify check
```

- 升級方式

```
uv tool install specify-cli --force --from git+https://github.com/github/spec-kit.git
```

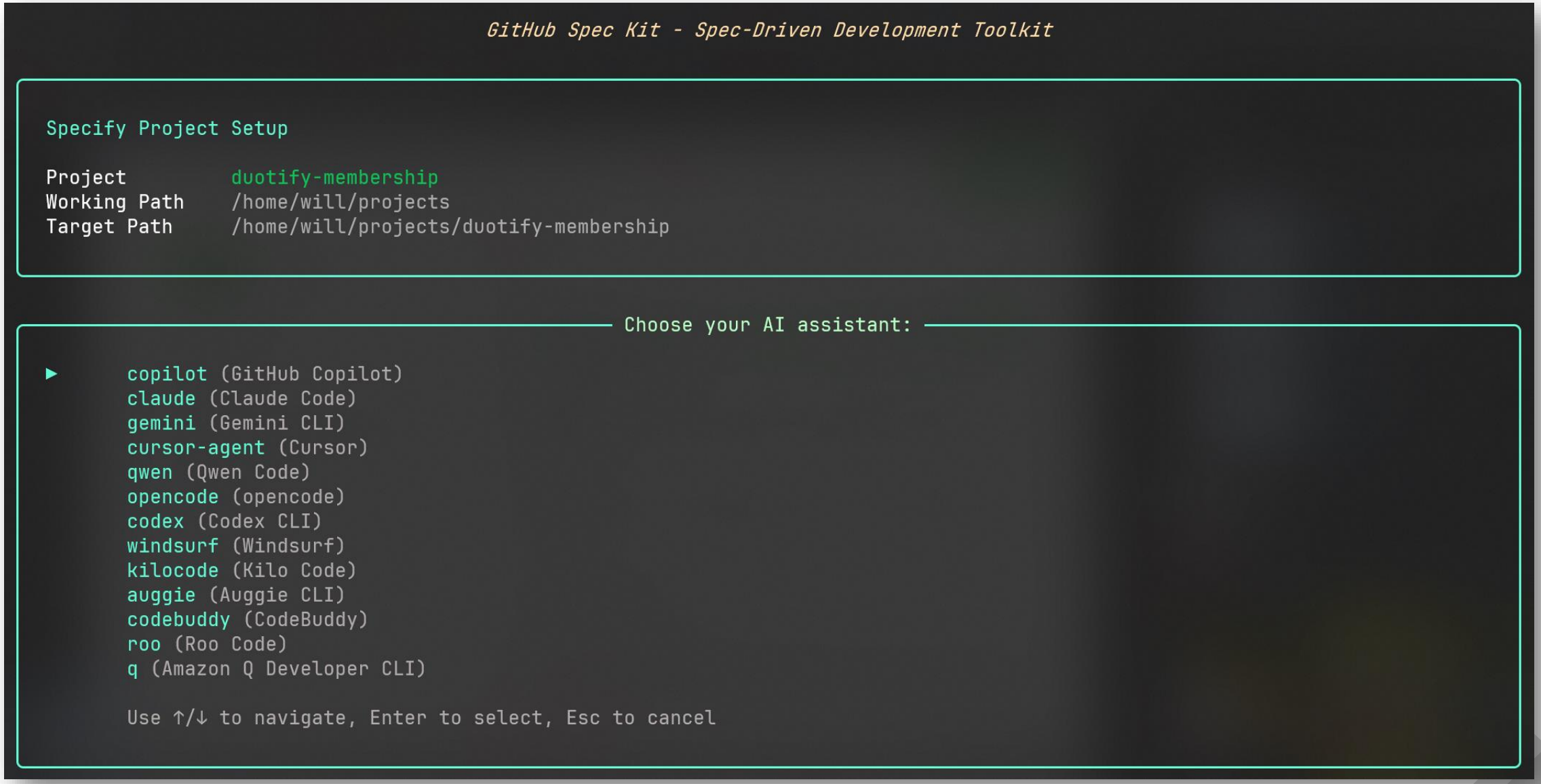
- 一次性使用 (One-time Usage)

```
uvx --from git+https://github.com/github/spec-kit.git specify init <PROJECT_NAME>
```

- 持久安裝的好處
  - 工具會持續安裝並可在 PATH 中使用，無需建立 shell 別名，讓 shell 設定更乾淨
  - 使用 `uv tool list`、`uv tool upgrade`、`uv tool uninstall` 可更好地管理工具

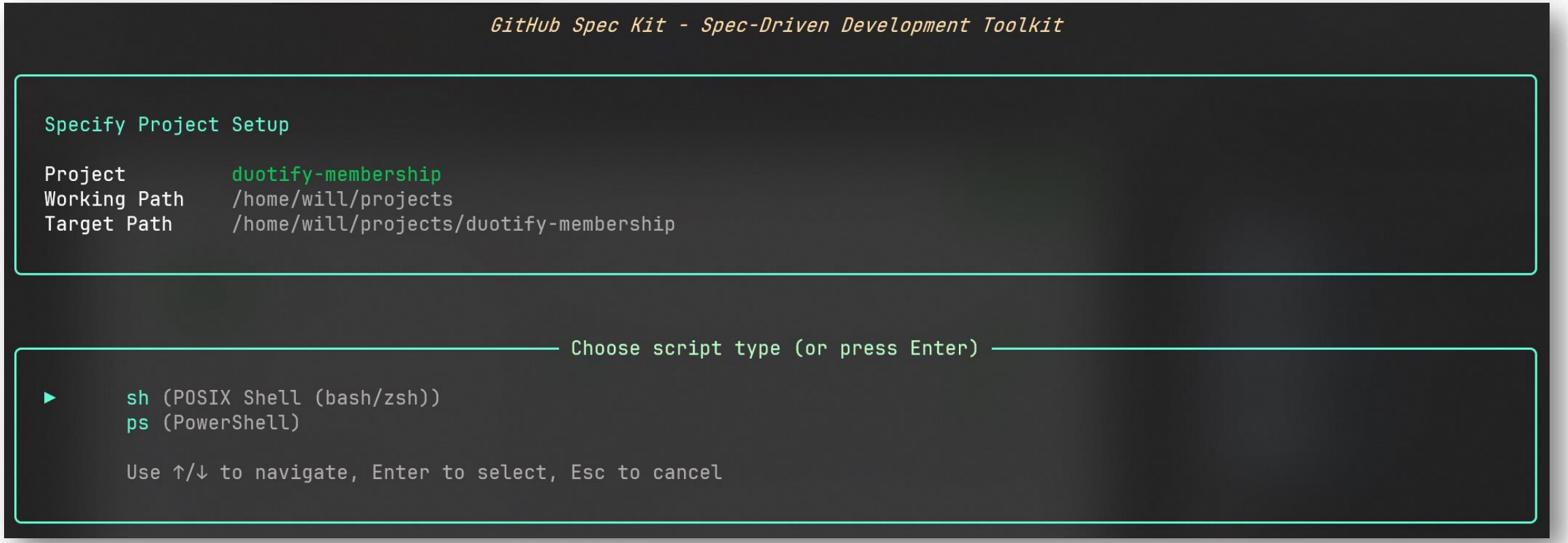
多奇·教育訓練

初始化全新專案（specify init NAME）



選擇腳本類型

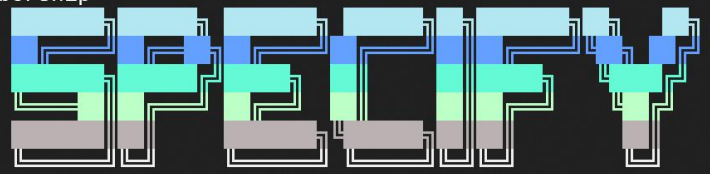
- Spec Kit 透過 **Shell script** 或 **PowerShell** 協助你完成 SDD 流程



多奇·教育訓練

第一次執行 specify init 會自動初始化專案

```
❯ specify init --ai copilot --script sh duotify-membership
```



GitHub Spec Kit - Spec-Driven Development Toolkit

```
Specify Project Setup

Project         duotify-membership
Working Path    /home/will/projects
Target Path     /home/will/projects/duotify-membership

Selected AI assistant: copilot
Selected script type: sh
Initialize Specify Project
├── ● Check required tools (ok)
├── ● Select AI assistant (copilot)
├── ● Select script type (sh)
├── ● Fetch latest release (release v0.0.62 (51,769 bytes))
├── ● Download template (spec-kit-template-copilot-sh-v0.0.62.zip)
├── ● Extract template
├── ● Archive contents (28 entries)
├── ● Extraction summary (3 top-level items)
├── ● Ensure scripts executable (5 updated)
├── ● Cleanup
├── ● Initialize git repository (initialized)
└── ● Finalize (project ready)

Project ready.
```


多奇·教育訓練

依據指示進行 AI 工具設定與 SDD 流程

```
Agent Folder Security

Some agents may store credentials, auth tokens, or other identifying and private artifacts in the agent folder within your
project.
Consider adding .github/ (or parts of it) to .gitignore to prevent accidental credential leakage.

Next Steps

1. Go to the project folder: cd duotify-membership
2. Start using slash commands with your AI agent:
   2.1 /speckit.constitution - Establish project principles
   2.2 /speckit.specify - Create baseline specification
   2.3 /speckit.plan - Create implementation plan
   2.4 /speckit.tasks - Generate actionable tasks
   2.5 /speckit.implement - Execute implementation

Enhancement Commands

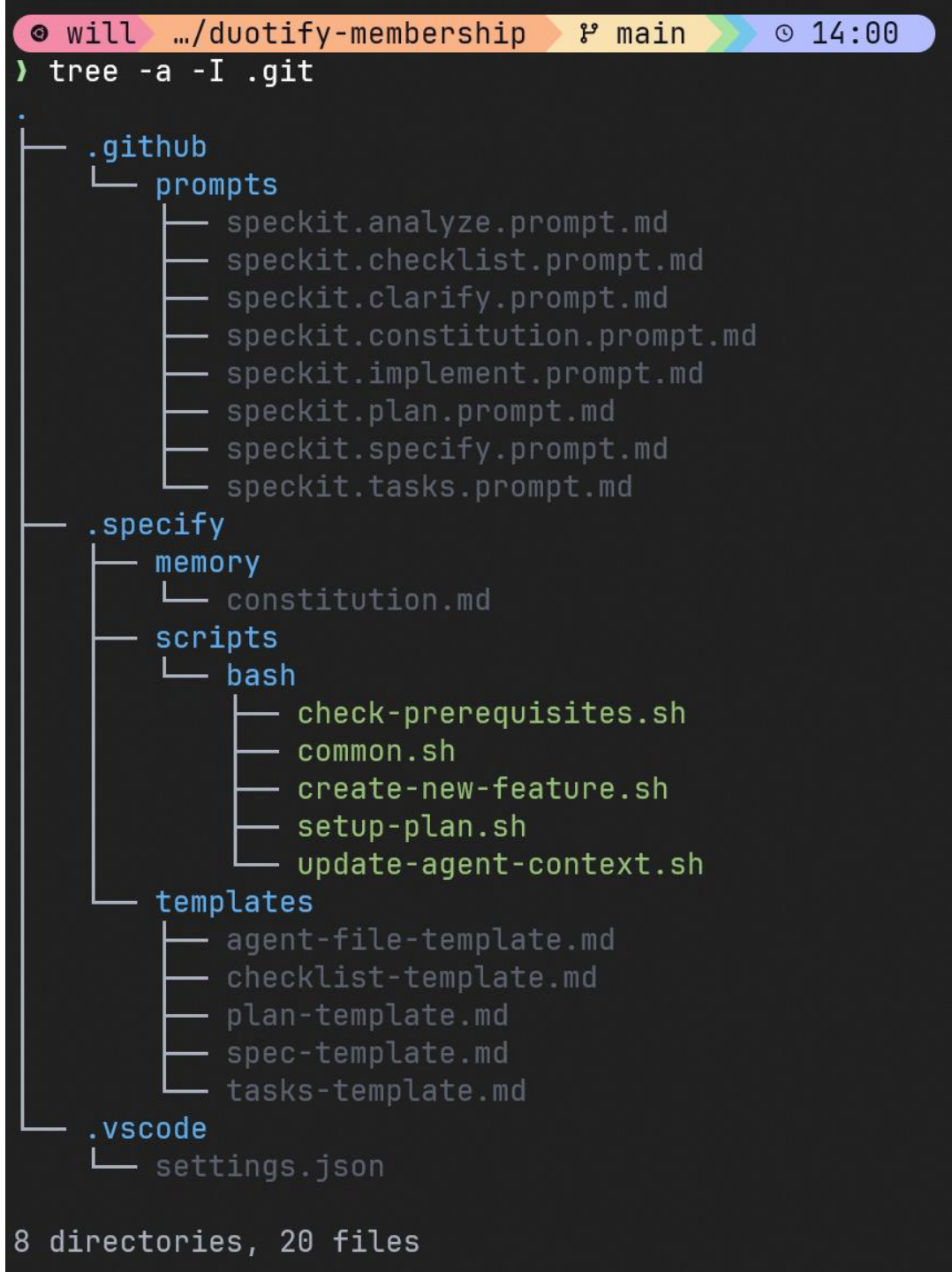
Optional commands that you can use for your specs (improve quality & confidence)

○ /speckit.clarify (optional) - Ask structured questions to de-risk ambiguous areas before planning (run before /speckit.plan
if used)
○ /speckit.analyze (optional) - Cross-artifact consistency & alignment report (after /speckit.tasks, before
/speckit.implement)
○ /speckit.checklist (optional) - Generate quality checklists to validate requirements completeness, clarity, and consistency
(after /speckit.plan)
```

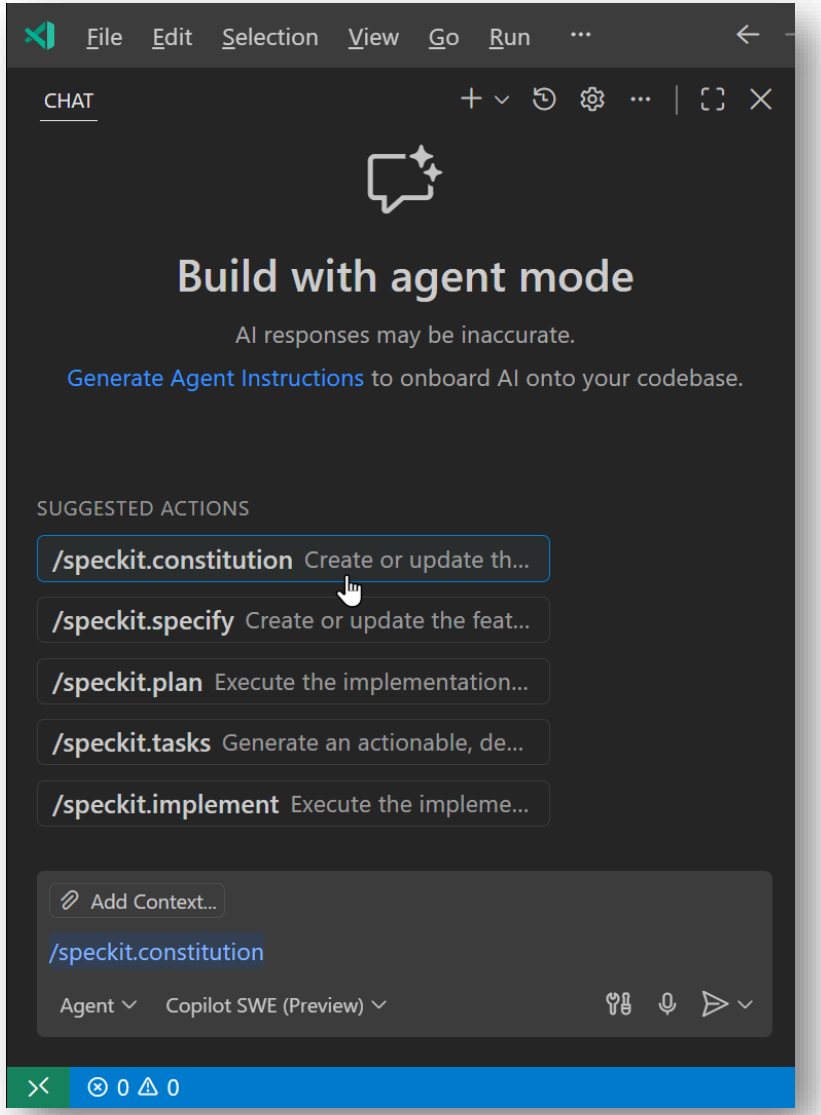

多奇·教育訓練

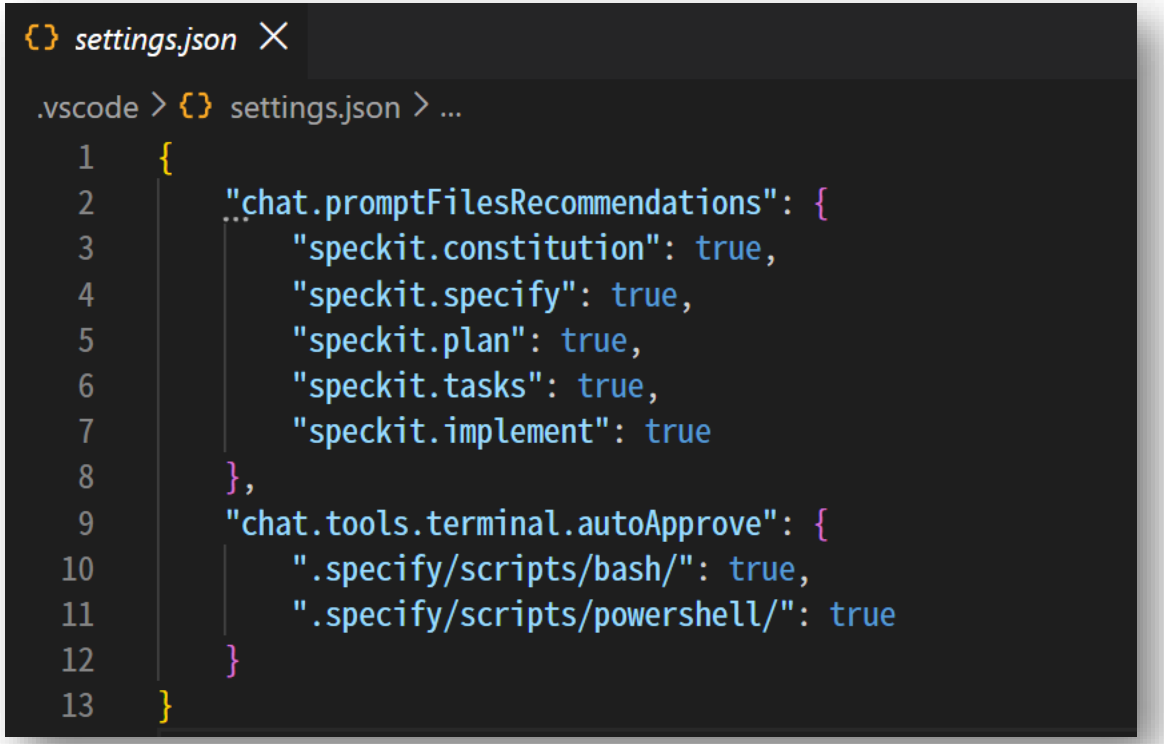
以 GitHub Copilot 為例

- 命令
  - **mkdir duotify-membership**
  - **specify init --ai codex --script sh --here**
- Spec Kit 的 SDD 流程範本與重要腳本
  - **.specify/***
- 斜線命令 (Slash commands)
  - **.github/prompts/***
- VS Code 相關設定
  - 含 **chat.tools.terminal.autoApprove** 自動審核設定
  - 含 **chat.promptFilesRecommendations** 提示檔案建議



目前 VS Code Insiders 有建議的提示功能





多奇·教育訓練

以 Codex CLI 為例

- 命令：**specify init --ai codex --script sh --debug duotify-membership**
- 需設定 **.gitignore** 刻意排除一些檔案加入版控，以免敏感資料外洩

```
.codex/
!.codex/prompts/
```

```
Project ready.

─ Agent Folder Security ─

Some agents may store credentials, auth tokens, or other identifying and private artifacts in
the agent folder within your project.
Consider adding .codex/ (or parts of it) to .gitignore to prevent accidental credential leakage.
```

- 目前需要額外設定 **CODEX_HOME** 環境變數才能使用

```
─ Next Steps ─

1. Go to the project folder: cd duotify-membership
2. Set CODEX_HOME environment variable before running Codex: export CODEX_HOME=/home/will/projects/duotify-membership/.codex
3. Start using slash commands with your AI agent:
   2.1 /speckit.constitution - Establish project principles
   2.2 /speckit.specify - Create baseline specification
   2.3 /speckit.plan - Create implementation plan
   2.4 /speckit.tasks - Generate actionable tasks
   2.5 /speckit.implement - Execute implementation
```


多奇·教育訓練

1 建立專案原則 (制定憲法)

- 使用 /speckit.constitution 指令來建立你的專案治理原則與開發指南，指引後續所有工作！

```
/speckit.constitution Create principles focused on code quality, testing standards, user experience consistency, and performance requirements
```

```
/speckit.constitution Create principles focused on
- Use TDD
- Use SOLID principal
- Use KISS principal
- Use ASP.NET Core 8.0 Web API
- Use Service to wrap common operations
- Use Repository to wrap common EF Core queries (CRUD)
- Use Git and Semantic versioning with Conventional Commits 1.0.0
- All API requests must be logged with ILogger<T>.
- All API request/response must have DTO classes as model.
```


多奇·教育訓練

2 建立初始規格

- 使用 `/speckit.specify` 指令來描述您想要建構的內容！
- 第一次執行會自動建立全新的 Git 分支，命名規則為 **###-feature-name** (預設不支援中文)
- 可以連續使用這個指令來更新現有規格，例如團隊在開會時，就可以當下即時更新規格！👍
- 請著重於**做什麼**(what)以及**為什麼**(why) 來建立「**使用者故事**」，可以先不用提及**技術堆疊**。

```
/speckit.specify Build an application that can help me organize my photos in
separate photo albums. Albums are grouped by date and can be re-organized by
dragging and dropping on the main page. Albums are never in other nested albums.
Within each album, photos are previewed in a tile-like interface.
```

```
/speckit.specify
Based on #file:PRD.md, I want to build 會員註冊 feature.
```

```
/speckit.specify
Update my spec with #file:01-會員註冊
```


多奇·教育訓練

建立初始規格的準備工作

- 建立初始規格時，建議先準備好 **PRD**（產品需求描述）文件
  - 你也可以說這是一份 **SA**（系統分析）文件

- 實務問題
  - 當使用者進行需求變更（CR），是否要回頭來修改規格？
  - 如果工程師判斷直接改 Code 比較快，那還要繼續修改規格嗎？

多奇·教育訓練

3 釐清規格中未明確的區域

- 使用 `/speckit.clarify` 指令來建釐清規格中**未明確**的區域
- 在建立好基準規格後，你可以針對第一次嘗試未能正確捕捉的需求進行釐清。
- 你應該在建立技術規劃之前，先執行結構化的釐清流程，以減少後續重工問題！
- 一次**最多五個**問題，想問幾次都可以！
  - Generate (internally) a prioritized queue of candidate clarification questions (maximum 5). Do NOT output them all at once.

🌟 **不要親手修改所有的規格！**

多奇·教育訓練

4 建立技術實作計畫

- 使用 `/speckit.plan` 指令以選定的技術堆疊建立技術實作計劃
- 這個步驟必須明確指定「**技術堆疊**」，這樣才能制定出**符合專案需求**的**技術規範**。

```
/speckit.plan We are going to generate this using .NET Aspire, using Postgres as the database. The frontend should use Blazor server with drag-and-drop task boards, real-time updates. There should be a REST API created with a projects API, tasks API, and a notifications API.
```

```
/speckit.plan
We are going to generate this using ASP.NET Core 8.0 Web API, using SQL Server as the database. This project is mainly for backend REST API only. No frontend implementation is required. Use EF Core Code First workflow.
```

```
I don't want to use AutoMapper to map DTO. Use POCO instead.
```

```
I don't want to use Redis.
```

不要親手修改所有的規格或計畫！

多奇·教育訓練

5 驗證計畫

- AI 生成的計畫，經常都有「**過度設計**」的問題，需要人工檢視！
- 讓 AI 依照計畫內容檢查一遍，可以確保計畫沒有遺漏任何部分。
- 以下提示詞可以幫你完整檢視計畫的完整度與潛在的缺失：

```
Now I want you to go and audit the implementation plan and the implementation
detail files.

Read through it with an eye on determining whether or not there is a sequence of
tasks that you need

to be doing that are obvious from reading this. Because I don't know if there's
enough here. For example,

when I look at the core implementation, it would be useful to reference the
appropriate places in the implementation

details where it can find the information as it walks through each step in the core
implementation or in the refinement.
```


多奇·教育訓練

6 將實作計畫拆解為任務清單

- 使用 `/speckit.tasks` 指令根據你的實作計畫建立可執行的任務清單。
- 這個步驟不需要再提供額外的提示詞

```
/speckit.tasks
```

- 它會根據當前的所有「**規格**」（**spec.md**）與「**計畫**」（**plan.md**）制定完整的任務清單
- 生成完畢之後，工程師必須詳細查看 **tasks.md** 檔案，確認每個步驟都合情合理
- 整理出「不合理」的任務，回頭看先前的「規格」與「計畫」是否有瑕疵！

不要親手修改所有的規格、計畫或任務！

多奇·教育訓練

7 進行跨產出物一致性與覆蓋率分析

- 使用 `/speckit.analyze` 指令可以針對 **Spec Kit 三大核心產出物**進行分析
  - **spec.md**
  - **plan.md**
  - **tasks.md**
- 自動辨識三份規格之間的**不一致**、**重複**、**模糊**及**規格不足**項目
- 自動偵測三份規格與 **constitution.md** 之間的衝突或矛盾的地方
- 這個 `/speckit.analyze` 指令只能在 /speckit.tasks 成功產出完整 **tasks.md** 後執行！

```
/speckit.analyze
```

> **矛盾的「上下文」是 AI 幻覺的主要來源！**

多奇·教育訓練

8 實作出完整程式碼

- 使用 /speckit.implement 來執行所有任務，並依照計畫建構你的功能。
- 從完整的「規格」中一口氣開發完畢，需要的是「**精準、合理的上下文**」支持！

```
/speckit.implement
```

```
繼續實作
```

```
繼續實作
```

```
繼續實作
```

💡 **請參考《透過 AI Agent 自動進行整個 SDD 開發流程》文件**

多奇·教育訓練

“

規格驅動開發的本質

就是

上下文工程

(Context Engineering)

”

多奇·教育訓練

Spec Kit 內建腳本說明 (PowerShell / Bash)

- **common.ps1**
  - 這是一個公共函式庫，為其他所有 PowerShell 腳本提供共享的輔助函式。不應該被直接執行，而是由其他腳本導入使用。
- **check-prerequisites.ps1**
  - 驗證當前功能分支的開發環境是否已正確設置，確保所有必需的文件和目錄都已就位。
  - 通常在執行其他開發工作流命令前調用。
- **create-new-feature.ps1**
  - 為新功能建立完整的開發環境，包括 **Git 分支**、**功能目錄結構**和**初始規格文件**。
- **setup-plan.ps1**
  - 為當前功能建立或初始化實施計畫文件 (plan.md)。這通常在**建立新功能**後立即執行。
- **update-agent-context.ps1**
  - 根據 **plan.md** 中的技術棧訊息，更新各種 AI 代理 (Claude、Gemini、GitHub Copilot、Cursor IDE 等) 的上下文文件。這確保所有 AI 工具都能了解項目的當前技術配置。

多奇·教育訓練

Spec Kit 自訂提示命令說明 (完整文件)

檔案名稱	執行階段	主要功能	輸出成果	前置條件
speckit.constitution.prompt.md	初始化	建立或更新專案憲法（治理原則）	constitution.md	無
speckit.specify.prompt.md	第一步	從自然語言特性描述生成規格書	spec.md requirements.md	無
speckit.clarify.prompt.md	第二步	識別規格書中的歧義並詢問澄清問題	更新後的 spec.md	spec.md 存在
speckit.plan.prompt.md	第三步	根據規格書生成實現計畫和設計文件	plan.md, data-model.md, contracts/, research.md, quickstart.md	spec.md 已確認
speckit.tasks.prompt.md	第四步	將計畫轉換為可執行的工作任務清單	tasks.md 含依賴關係	plan.md, spec.md 存在
speckit.checklist.prompt.md	並行/重複	為特定領域生成需求質量檢查清單	checklists/[domain].md	spec.md 存在
speckit.analyze.prompt.md	第五步	對三個核心文件進行交叉一致性分析	分析報告 (無檔案修改)	spec.md, plan.md, tasks.md 全部存在
speckit.implement.prompt.md	第六步	執行任務清單中的所有實現任務	已實現的功能/測試/文件	tasks.md, plan.md 存在，檢查清單完成

多奇·教育訓練

結語

- 實作只是延伸，理解才是主體
- Spec Kit 不是工具，是一種紀律
- 我們不再追快，而是讓快有方向
- 技術決策有據可查，創意也有邏輯
- AI 幫我們寫程式，但意圖還是人的
- 讓 SDD 把團隊的共識，轉成可執行的語言
- 當規格能驅動程式碼，程式碼就不再驅動我們

多奇·教育訓練

相關連結

- 學員課前注意事項
  - https://sdd.gh.miniasp.com/
- Spec Kit 官網
  - https://github.com/github/spec-kit/
- Spec Kit 正體中文版
  - https://github.com/doggy8088/spec-kit/
  - https://github.com/doggy8088/spec-kit/wiki (版本變更追蹤)
  - https://github.com/doggy8088/spec-kit/blob/zh-tw/spec-driven.md
- 教學影片
  - https://www.youtube.com/@DenDev (作者 Den Delimarsky 是 Spec Kit 主要維護者)
- 文章分享
  - 從 SBE 到 SDD：讓 AI 聽得懂人話的開發新思維 - 轉念學 - 敏捷三叔公的學習之旅

多奇·教育訓練



聯絡資訊

The Will Will Web

網路世界的學習心得與技術分享

https://blog.miniasp.com

Facebook

Will 保哥的技術交流中心

https://www.facebook.com/will.fans

Twitter

https://x.com/Will_Huang

多奇·教育訓練



多奇·教育訓練

THANK YOU!

Q&A